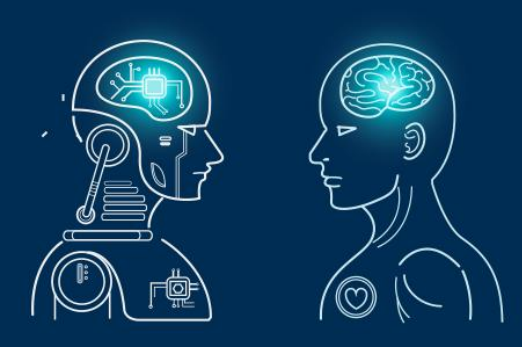




Cátedra de INTELIGENCIA ARTIFICIAL

Sistemas Basados en Conocimientos Los Sistemas Expertos

Mag. Ing. Mario M. Figueroa de la Cruz



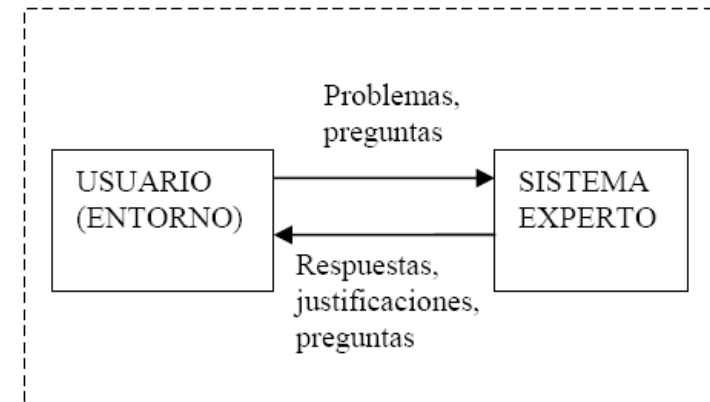
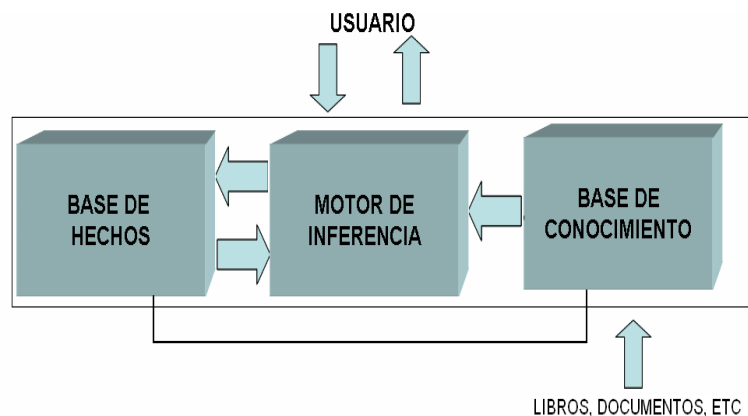
DEFINICION

SISTEMA EXPERTO (SE)

“Programa computacional que exhibe, dentro de un dominio específico, un grado de experiencia en la solución de un problema comparable con la forma en que un experto humano lo haría.”

Definiciones

Los SBC plantean que la posibilidad de resolver determinados problemas radica en un “saber” o conocimiento específico sobre el tipo de problemas mas que en capacidades intelectivas generales.



SISTEMAS EXPERTOS

El termino Sistema Experto se reserva para los programas cuyo conocimiento proviene de un experto humano, para diferenciarlos de los programas cuyo conocimiento proviene de los libros de texto o de los no expertos.

Características

- Se basan principalmente en procesos heurísticos antes que algorítmicos.
- Los SE involucran representación de conocimiento heurístico (amplio, impreciso, mal definido al alcance de pocos expertos)
- Se debe utilizar técnicas de transferencia denominada Adquisición del Conocimiento
- La naturaleza y cantidad de conocimiento para resolver sistemas convencionales puede ser bien estimada, no así en los SE
- Pueden tomar decisiones aunque el conocimiento sea **incierto, impreciso o incompleto**.



Características

- ☐ Amplio conocimiento del dominio.
- ☐ Soporte para análisis heurística.
- ☐ Procesamiento de símbolos.
- ☐ Habilidad para inferir nuevos conocimientos a partir de conocimientos ya existentes, Aplicando técnicas de búsqueda heurística.
- ☐ Capacidad para explicar su propio razonamiento.

Porque los Sistemas Expertos?



- Necesidades económicas
 - Disponer de expertos humanos altamente cualificados es caro y no siempre posible.
 - Uso como herramientas “baratas” de aprendizaje para otros expertos y no expertos.
 - Preservación del conocimiento de los expertos.
- Necesidades de eficiencia computacional
 - Los métodos generales de resolución de problemas complejos presentan limitaciones (métodos débiles).

Aplicabilidad



- Muy eficaces en tareas de análisis, diagnósticos, clasificación, interpretación.
- Con capacidad de autoexplicación.
- Con facilidad de comunicación con el usuario.
- “Fácilmente” extensibles.
- Con capacidad de razonamiento aproximado.

V E N T A J A S

EXPERIENCIA HUMANA

EXPERIENCIA ARTIFICIAL

Perecedero

Permanente

Difícil de transferir

Fácil de transferir

Difícil de documentar

Fácil de documentar

No predecible

Consistente

Costoso

Disponible

DESVENTAJAS

EXPERIENCIA HUMANA	EXPERIENCIA ARTIFICIAL
Creatividad	Sin inspiración
Es adaptativa	Ambiente fijo
Experiencia sensorial	Entrada simbólica
Enfoque amplio	Enfoque específico
Sentido común	Técnico

DIFERENCIAS ENTRE UN SE Y UN PROGRAMA CONVENCIONAL



PROCESAMIENTO DE DATOS	INGENIERÍA DEL CONOCIMIENTO
Representación y uso de datos	Representación y uso de conocimiento
Algoritmos	Heurísticas
Procesos repetitivos	Procesos inferenciales
Manipulación efectiva de bases de datos	Manipulación efectiva de bases de conocimiento



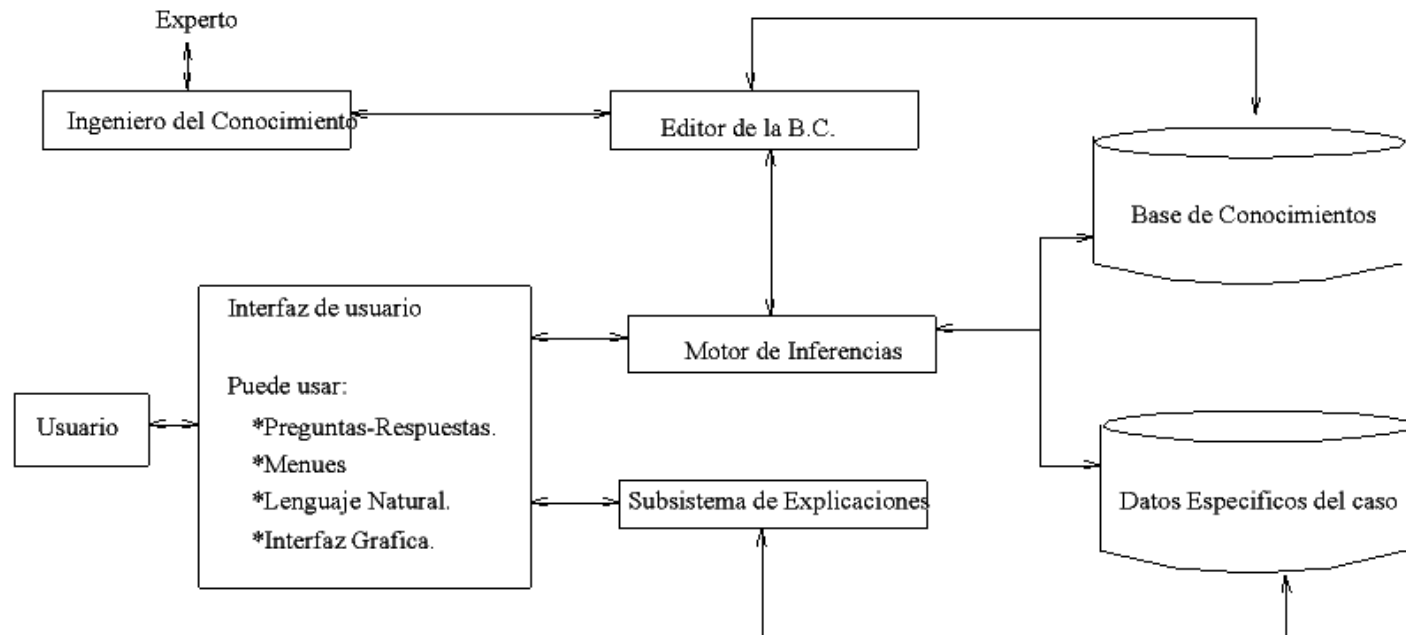
Primeros Sistemas Expertos

- DENDRAL (1965-1970)
 - Interpretación de espectrografía de masa y resonancia magnética de moléculas orgánicas
- MYCIN (1972-1976)
 - Diagnósis de enfermedades infecciosas en la sangre
 - 400 reglas (Razonamiento con incertidumbre)
- EMYCIN (1980)
 - Generaliza el sistema de control de MYCIN
 - Primer entorno de SE (shell)
- PROSPECTOR (1977)
 - Prospecciones mineras (Usa un método de razonamiento con incertidumbre)
- R1/XCON (1980) @ DEC
 - Sistema de configuración de entornos computacionales en *Digital Equipment Corporation*
 - 200.000 reglas
- INTERNIST (1982)
 - Diagnóstico en medicina interna
 - 1.000.000 reglas

ARQUITECTURA BÁSICA DE LOS SISTEMAS EXPERTOS



ARQUITECTURA DE UN SISTEMA EXPERTO



ARQUITECTURA BÁSICA DE LOS SISTEMAS EXPERTOS



- Base de conocimientos. Es la parte del sistema experto que contiene el conocimiento sobre el dominio. hay que obtener el conocimiento del experto y codificarlo en la base de conocimientos. Una forma clásica de representar el conocimiento en un sistema experto son las reglas. Una regla es una estructura condicional que relaciona lógicamente la información contenida en la parte del antecedente con otra información contenida en la parte del consecuente.
- Base de hechos (Memoria de trabajo). Contiene los hechos sobre un problema que se han descubierto durante una consulta. Durante una consulta con el sistema experto, el usuario introduce la información del problema actual en la base de hechos. El sistema empareja esta información con el conocimiento disponible en la base de conocimientos para deducir nuevos hechos.

ARQUITECTURA BÁSICA DE LOS SISTEMAS EXPERTOS



- Motor de inferencia. El sistema experto modela el proceso de razonamiento humano con un módulo conocido como el motor de inferencia. Dicho motor de inferencia trabaja con la información contenida en la base de conocimientos y la base de hechos para deducir nuevos hechos. Contrasta los hechos particulares de la base de hechos con el conocimiento contenido en la base de conocimientos para obtener conclusiones acerca del problema.
- Subsistema de explicación. Una característica de los sistemas expertos es su habilidad para explicar su razonamiento. Usando el módulo del subsistema de explicación, un sistema experto puede proporcionar una explicación al usuario de por qué está haciendo una pregunta y cómo ha llegado a una conclusión. Este módulo proporciona beneficios tanto al diseñador del sistema como al usuario. El diseñador puede usarlo para detectar errores y el usuario se beneficia de la transparencia del sistema.

ARQUITECTURA BÁSICA DE LOS SISTEMAS EXPERTOS



- Interfaz de usuario. La interacción entre un sistema experto y un usuario se realiza en lenguaje natural. También es altamente interactiva y sigue el patrón de la conversación entre seres humanos. Para conducir este proceso de manera aceptable para el usuario es especialmente importante el diseño del interfaz de usuario. Un requerimiento básico del interfaz es la habilidad de hacer preguntas. Para obtener información fiable del usuario hay que poner especial cuidado en el diseño de las cuestiones. Esto puede requerir diseñar el interfaz usando menús o gráficos.

Representación del Conocimiento



Necesitamos formalismos para representar el conocimiento que:

- *Sean utilizables en un computador*
- *Resulten coherentes con la manera que pensamos lo internaliza un especialista*
- *Faciliten las operaciones de recolección,*
- *organización, mantenimiento, validación ...*

... transparentes



Representación del Conocimiento

Algunos formalismos:

Reglas Heurísticas

Triplos OAV

Redes Semánticas

Marcos (FRAMES)

Fórmulas Lógicas

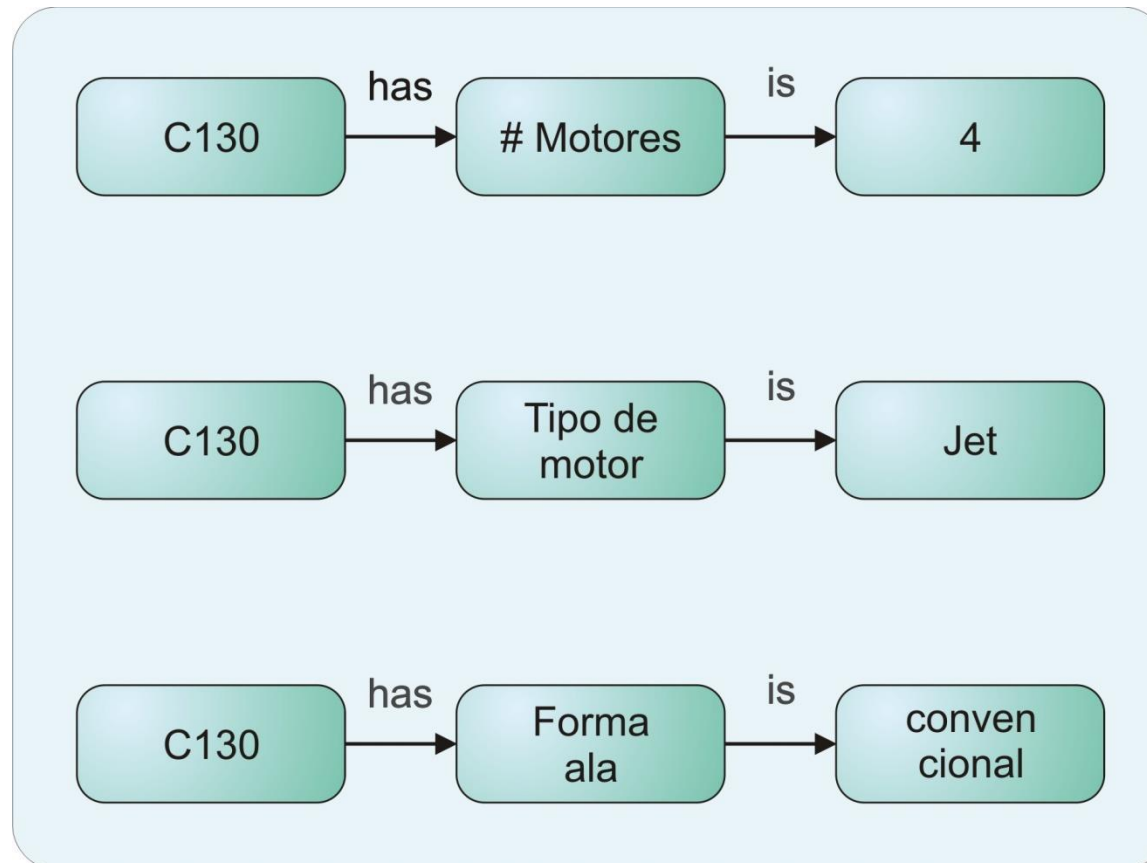
Redes Neuronales

Representación del Conocimiento

Triplos Objeto-Atributo-Valor

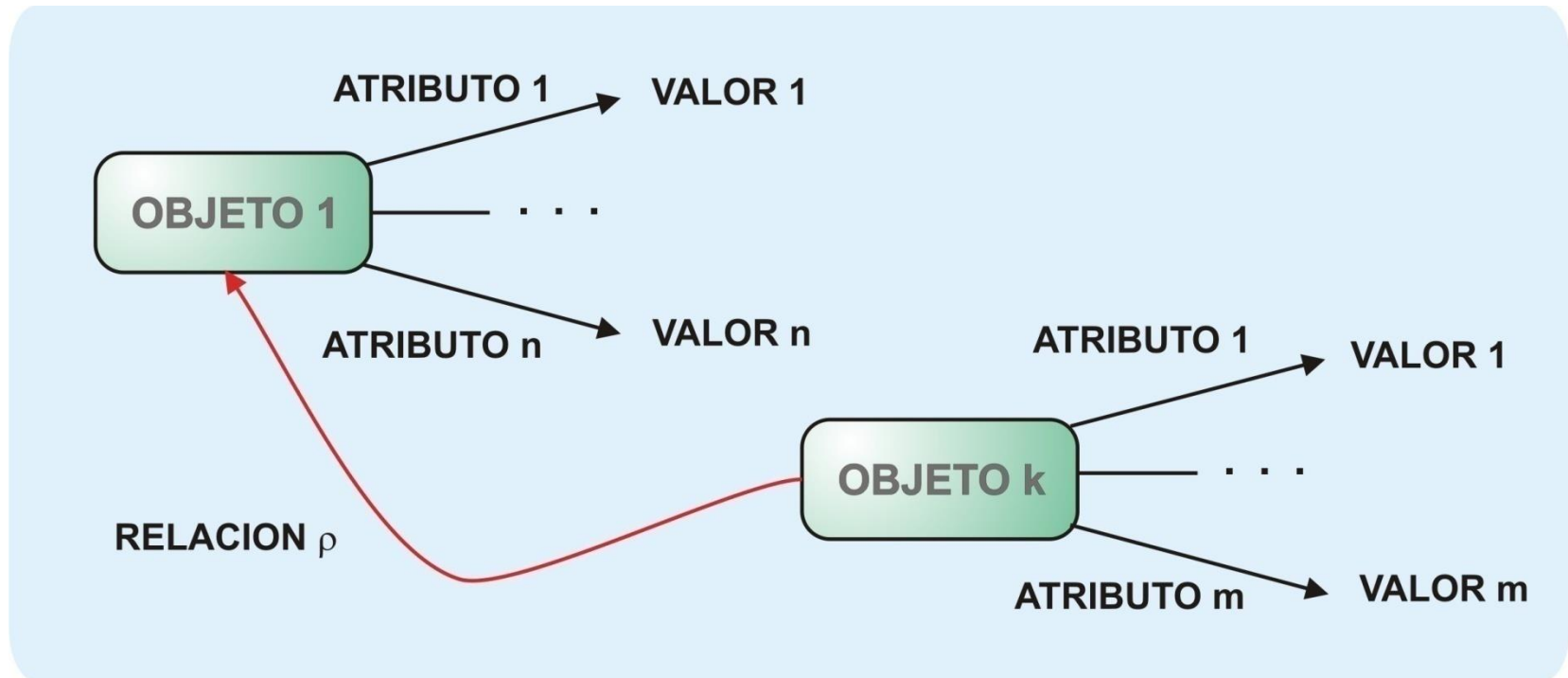


Representación del Conocimiento



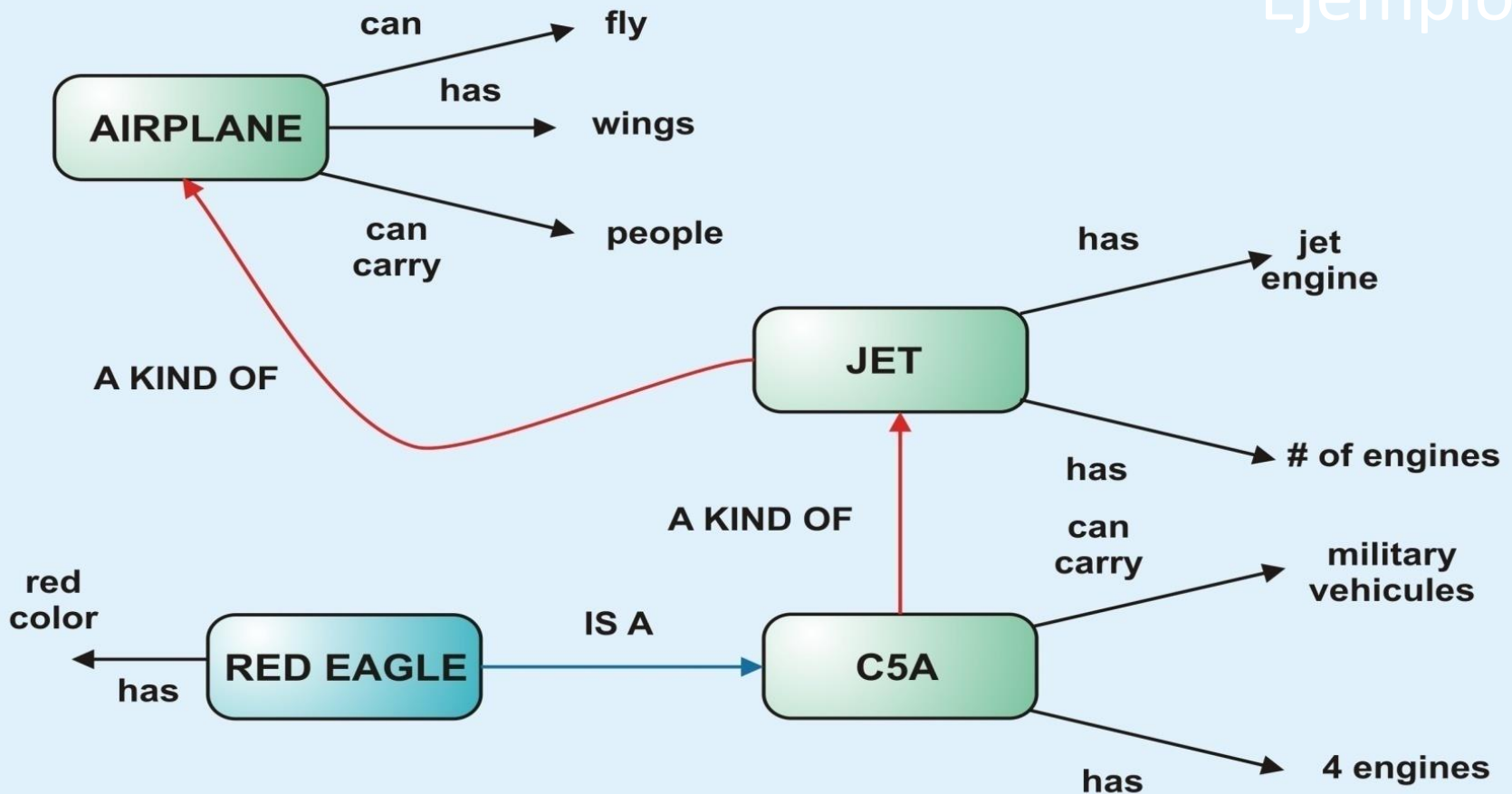
Representación del Conocimiento

Redes Semánticas

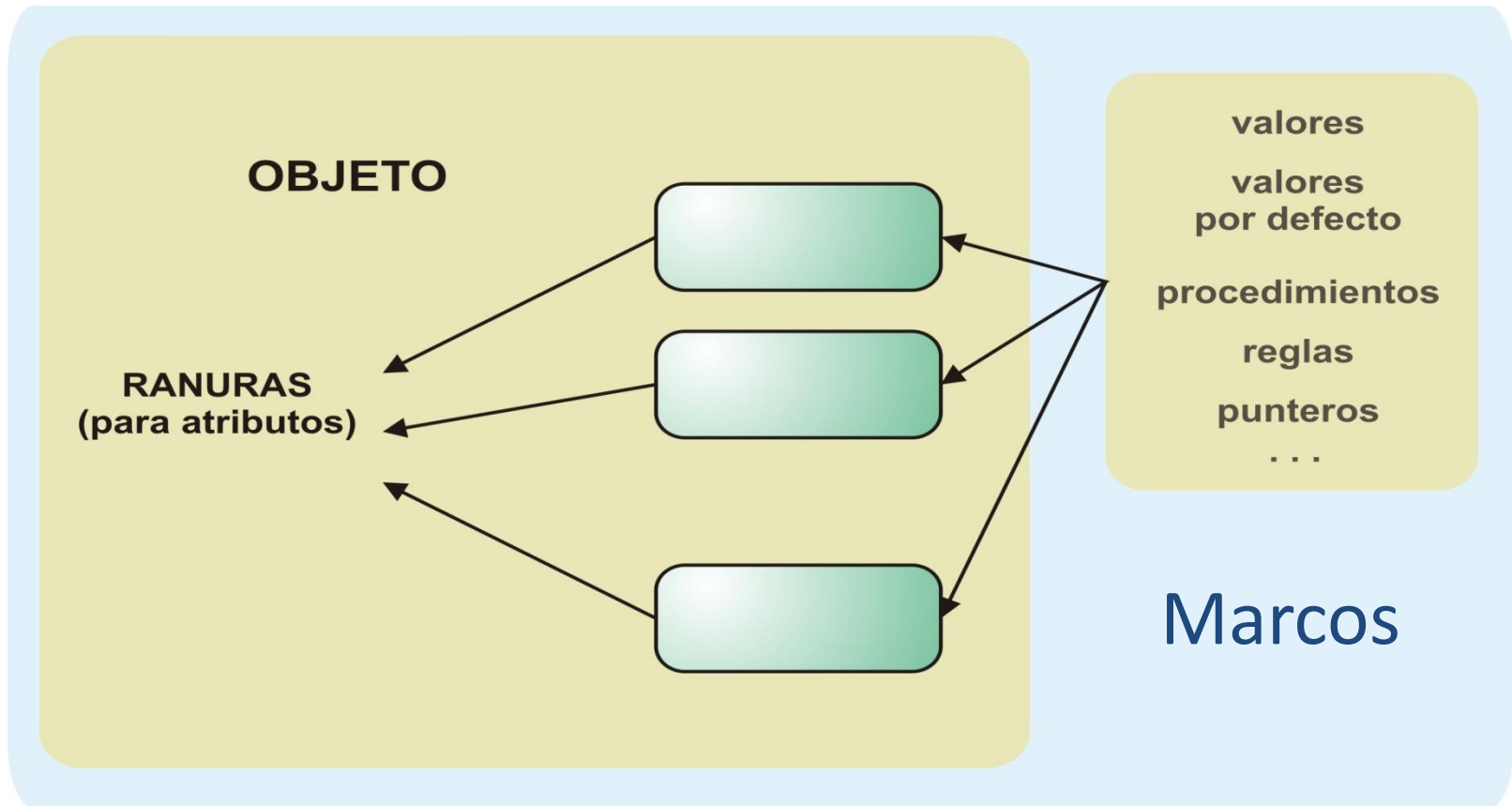


Representación del Conocimiento

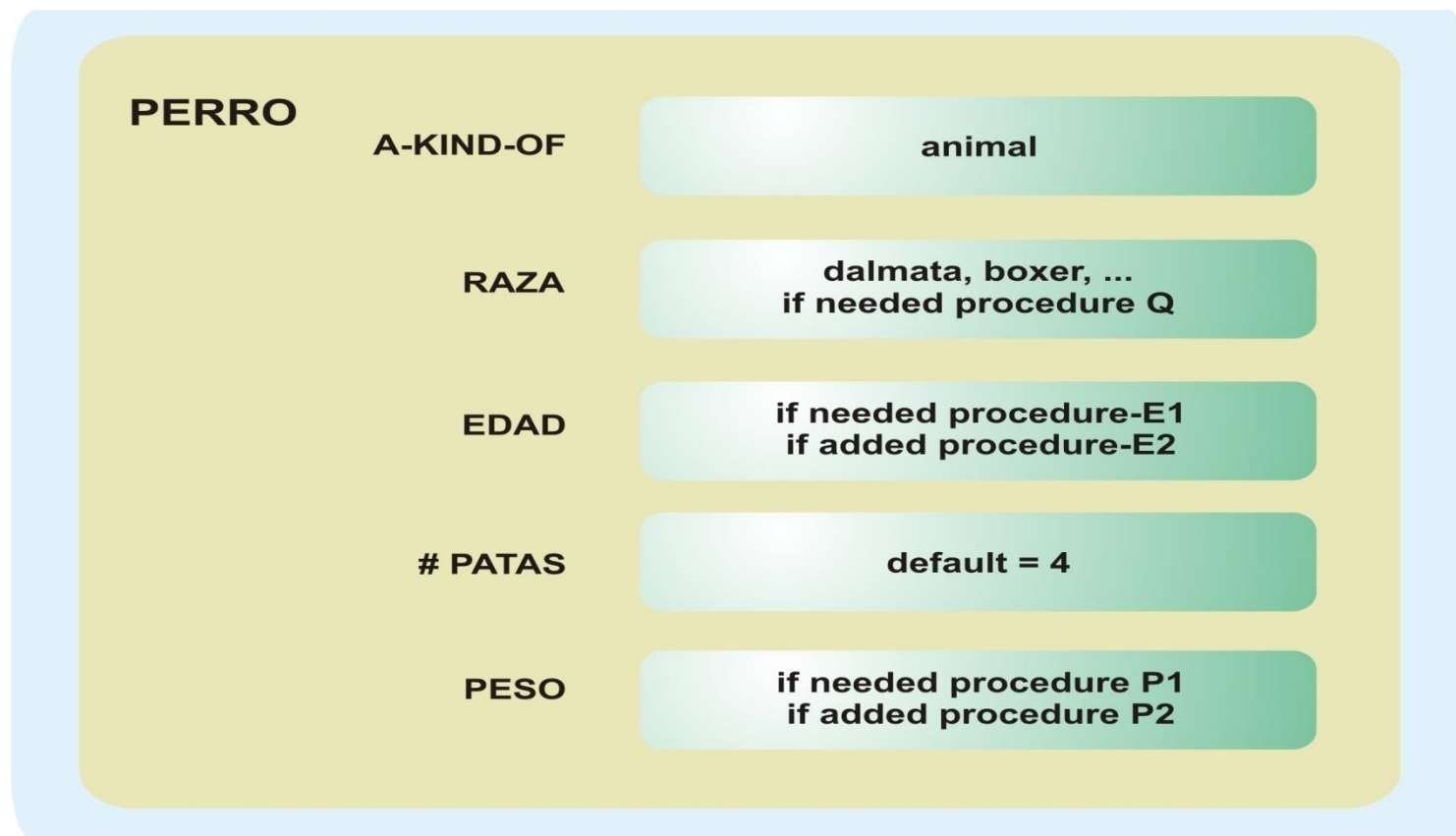
Ejemplo:



Representación del Conocimiento



Representación del Conocimiento



Representación del Conocimiento

Marcos (Frames)

Adecuados para sistemas complejos, de gran escala, involucrando valores por defecto y cantidades elevadas de datos conocidos a-priori

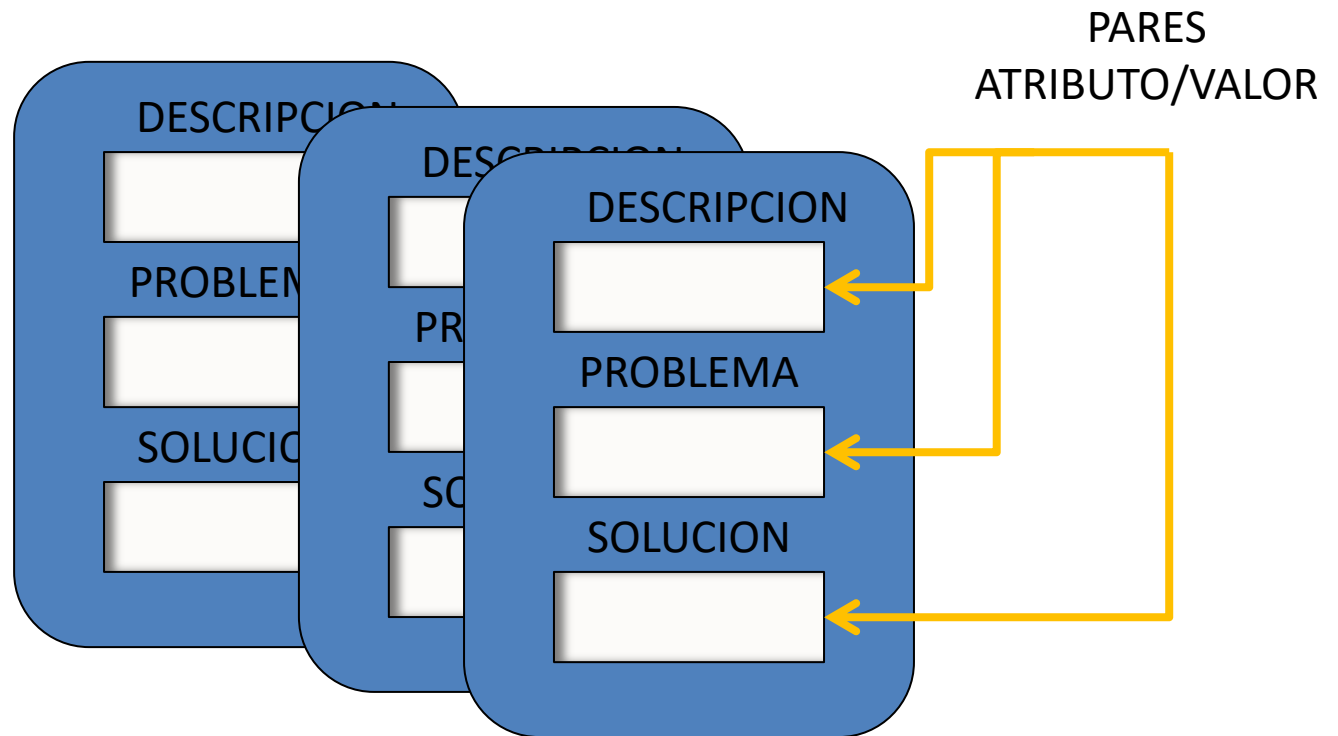
<i>Pros</i>	<i>Contras</i>
<i>Poderosos</i>	<i>Difícil de prever</i>
<i>Robustos</i>	<i>Comportamiento</i>
<i>Versátiles</i>	
<i>Expresivos</i>	

Adecuados para sistemas complejos, de gran escala, involucrando valores por defecto y cantidades elevadas de datos conocidos a-priori

Representación del Conocimiento



Casos



Representación del Conocimiento

DESCRIPCION

Tipo de Caso: El puerto I/O no está inicializado correctamente
La impresora no imprime, Autotest OK

PROBLEMA

Preguntas:
La impresora imprime una hoja de autotest? SI
Mensaje en display? O3 I/O problem
Puede imprimir desde el computador? NO
El puerto de I/O trabaja con otros dispositivos? NO

SOLUCION

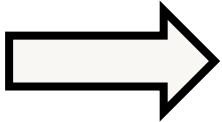
Acción:
Consultar manual del computador para inicializar el puerto I/O adecuadamente

Representación del Conocimiento



Reglas

Modelo psicológico de comportamiento humano (*reglas de producción*):

Estímulos  Acciones

En este modelo, un *procesador cognitivo* intenta disparar las reglas activadas por estímulos adecuados

Representación del Conocimiento

Forma Usual

IF <antecedente> **THEN** <consecuente>1
ELSE <consecuente2>

O bien:

IF <condición1> AND ... AND <condiciónN>
THEN <conclusión11> AND ... AND <conclusión1M>
ELSE <conclusión21> AND ... AND <conclusión2K>



Representación del Conocimiento

Regla 1:

IF sensor A toma un valor > 50
THEN temperatura del agua es bastante alta
ELSE temperatura del agua es normal

Alternativa (de mas fácil validación):

Regla 1a: IF sensor A toma un valor > 50
THEN temperatura del agua es bastante alta
Regla 1b: IF sensor A toma un valor ≤ 50
THEN temperatura del agua es normal

Representación del Conocimiento



Forma General

IF condición1 OP ... OP condiciónN
THEN conclusión1 OP ... conclusiónM

Donde OP indica un Operador Lógico,
típicamente AND o bien OR.

Equivalencia de términos:

premisa  condición

Representación del Conocimiento

Reglas vs. Triplos OAV

Regla 1a:

IF sensor A toma un valor > 50

THEN temperatura del agua es bastante alta

OBJETO
ATRIBUTO
VALOR



IF		THEN	
SENSOR A		AGUA	
TOMA UN VALOR		TEMPERATURA	
> 50		BASTANTE ALTA	



Representación del Conocimiento

Otras formas de Representación :

- Lógica de Predicados
- Redes Neuronales

SISTEMAS BASADOS EN EL CONOCIMIENTO



Inferencia como búsqueda

Proceso de búsqueda:

- Conjunto de soluciones potenciales
- Conjunto de restricciones

Sistemas basados en reglas:

- Encadenamiento de reglas

SISTEMAS BASADOS EN EL CONOCIMIENTO



ENCADENAMIENTO

Dos métodos básicos:

- ❖ Encadenamiento hacia adelante (Forward Chaining) o Inferencia Ascendente:

Orientado por HECHOS

Partiendo de un conjunto de hechos, usar reglas para deducir nuevos hechos

- ❖ Encadenamiento hacia atrás (Backward Chaining) o Inferencia Descendente:

Orientado por OBJETIVOS

Partiendo de un objetivo, usar reglas para buscar soporte del objetivo

SISTEMAS BASADOS EN EL CONOCIMIENTO



Encadenamiento hacia adelante

- Dado un conjunto de Hechos, elegir una regla cuyo Antecedente sea verdadero; disparar la regla;
- el Consecuente de la regla es un nuevo hecho;
- Continuar (recursivamente) con un nuevo conjunto de datos;
- Parar cuando no se pueda disparar ninguna regla.

SISTEMAS BASADOS EN EL CONOCIMIENTO



Encadenamiento hacia atrás

- Dado un Objetivo, parar si existe un Hecho que lo soporte;
- Caso contrario, seleccionar una regla cuyo Consecuente pueda soportar al Objetivo;
- Considerar las condiciones del Antecedente de la regla como nuevos Objetivos, y continuar (recursivamente)

SISTEMAS BASADOS EN CONOCIMIENTO



Forward y Backward Chaining

HACIA ADELANTE	HACIA ATRAS
Orientada por datos	Orientada por objetivos
Busca soluciones que sean consecuencia de los hechos	Busca hechos que soporten hipótesis
Tiende a ser mas eficiente en problemas con pocas condiciones posibles	Tiende a ser mas eficiente en problemas con pocas conclusiones posibles

SISTEMAS BASADOS EN EL CONOCIMIENTO



Encadenamiento mixto

Usar alternadamente los dos mecanismos durante una misma sesión de resolución de problemas

Ejemplos de estrategias:

Prioridad al encadenamiento para atrás:

- Usar encadenamiento hacia adelante solo cuando no es posible hacerlo con encadenamiento para atrás.

SISTEMAS BASADOS EN EL CONOCIMIENTO



Prioridad al encadenamiento hacia adelante:

- Usar encadenamiento hacia atrás solo cuando no sea posible hacerlo con encadenamiento hacia adelante

Modo dependiente de la fase de resolución del problema:

- Cambiar el modo de acuerdo con las características de cada fase

SISTEMAS BASADOS EN EL CONOCIMIENTO



Generación de Explicaciones

Una de las características más interesantes de los sistemas basados en reglas es el de poder entregar a pedido del usuario una explicación de cómo llegó al resultado que le entrega. Esta característica muy humana hace de estos sistemas los preferidos para realizar una gran cantidad de aplicaciones.

SISTEMAS BASADOS EN EL CONOCIMIENTO

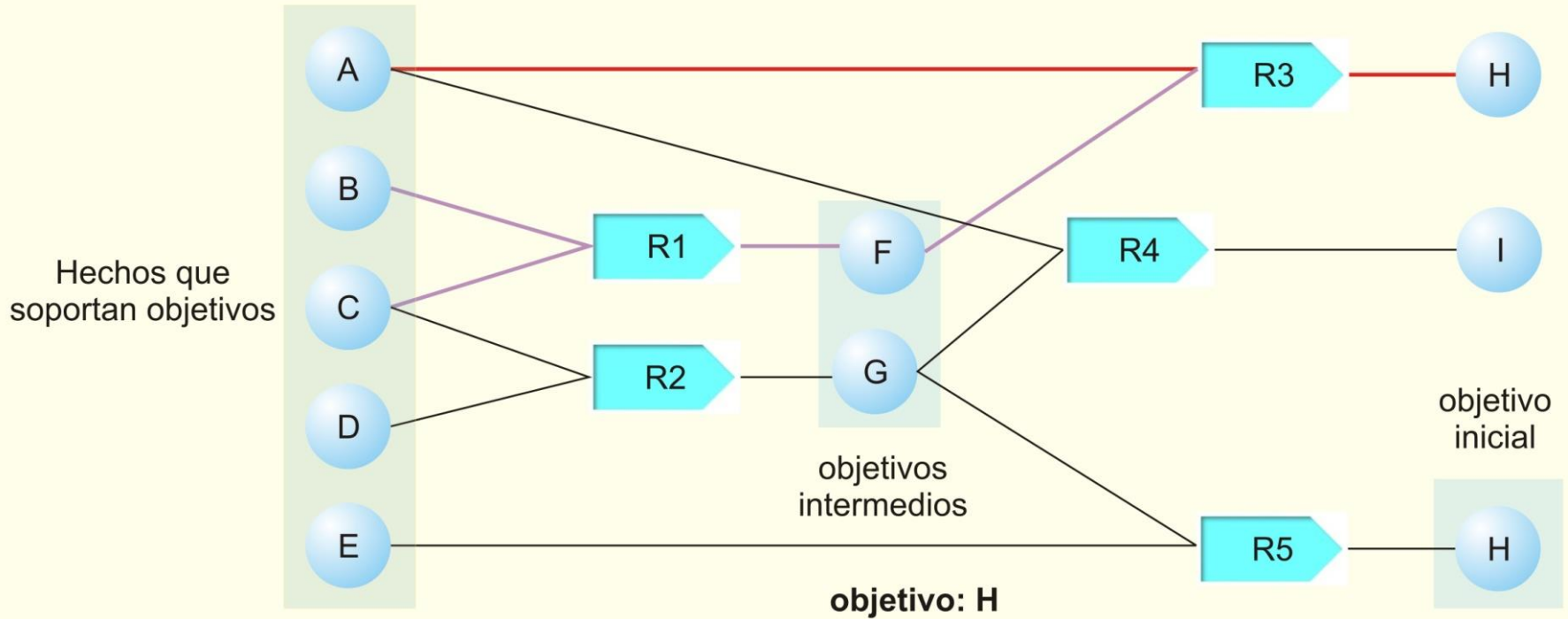


Explicaciones How:

Traza del proceso de encadenamiento

- Objetivo Inicial: H
- Pregunta: ¿Como eligió H?
- La explicación debe ser sobre todas las posibilidades consideradas

SISTEMAS BASADOS EN EL CONOCIMIENTO



SISTEMAS BASADOS EN EL CONOCIMIENTO



- Usuario: ¿Como eligió H?
- El sistema:

H es verdadero por que

fue inferido de R3

disparada por el hecho A que es TRUE

F es TRUE por que

fue inferido de R1

disparada por que el hecho B es TRUE

C es un hecho TRUE

SISTEMAS BASADOS EN EL CONOCIMIENTO



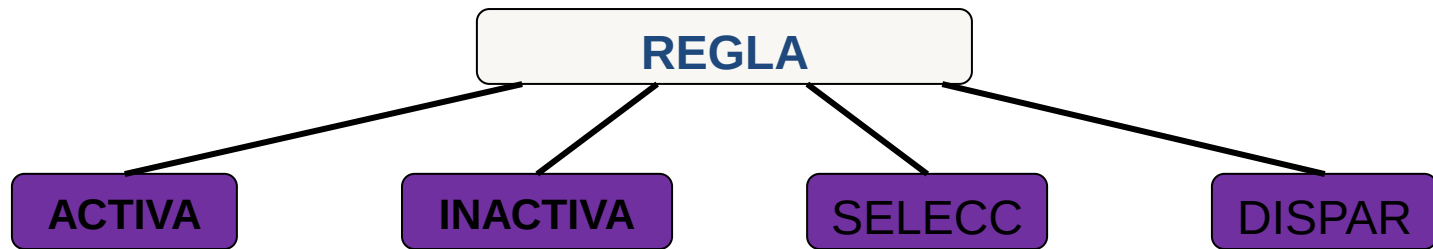
Organización de Reglas

- **Agrupamiento:**
 - por conclusiones con atributos similares
- **Ordenamiento**
 - ordenar los grupos de forma que resulten en los primeros lugares aquellos cuyas conclusiones estén mas próximas a los objetivos finales

SISTEMAS BASADOS EN EL CONOCIMIENTO



Estados de una Regla



Regla Seleccionada:

- El antecedente tiene valor TRUE

Regla Inactiva:

- El antecedente tiene valor FALSE

SISTEMAS BASADOS EN EL CONOCIMIENTO



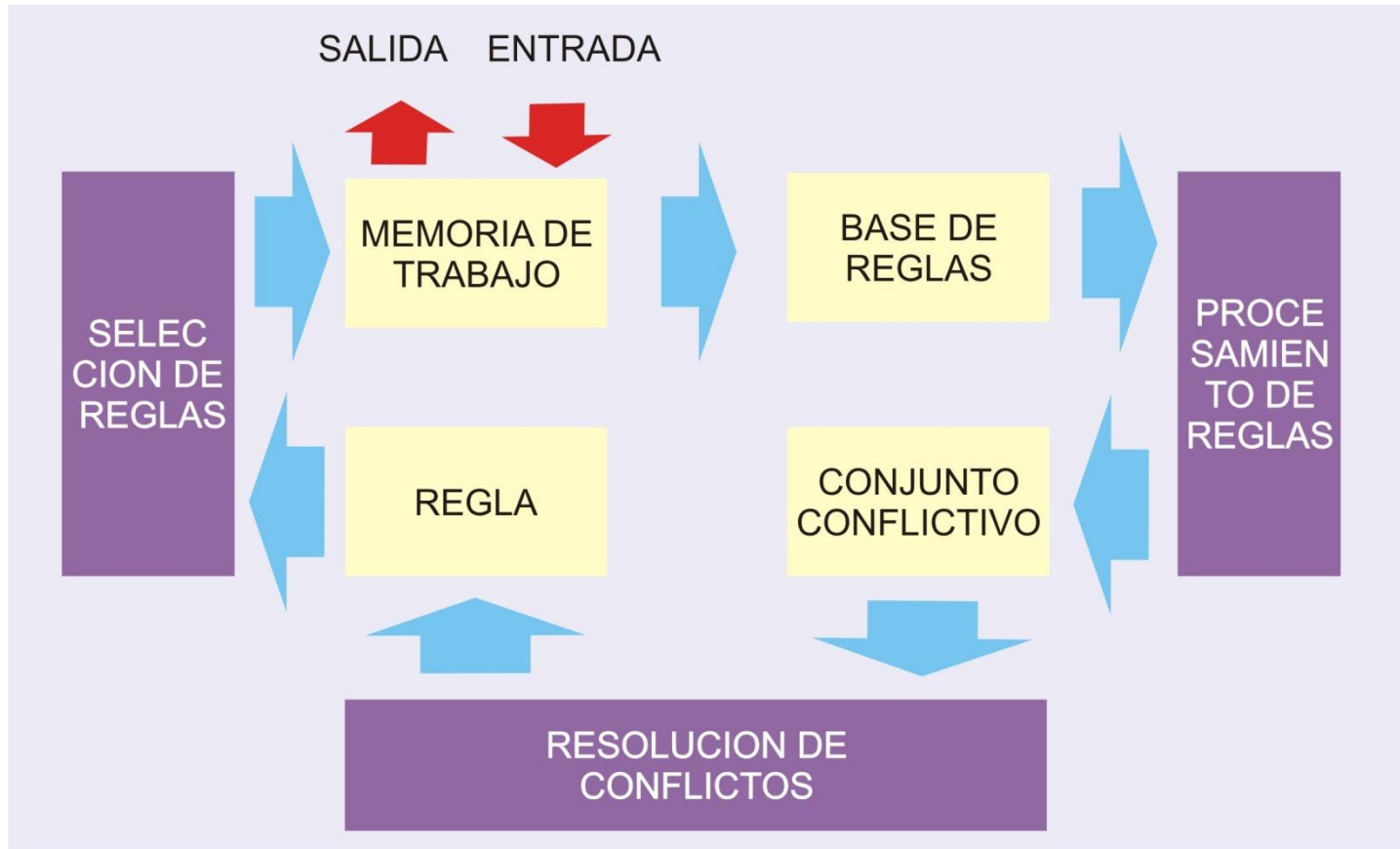
Regla Disparada:

- Regla cuyas acciones en el consecuente fueron ejecutadas

Regla Activa:

- Regla que no está Inactiva ni fue Disparada

SISTEMAS BASADOS EN EL CONOCIMIENTO

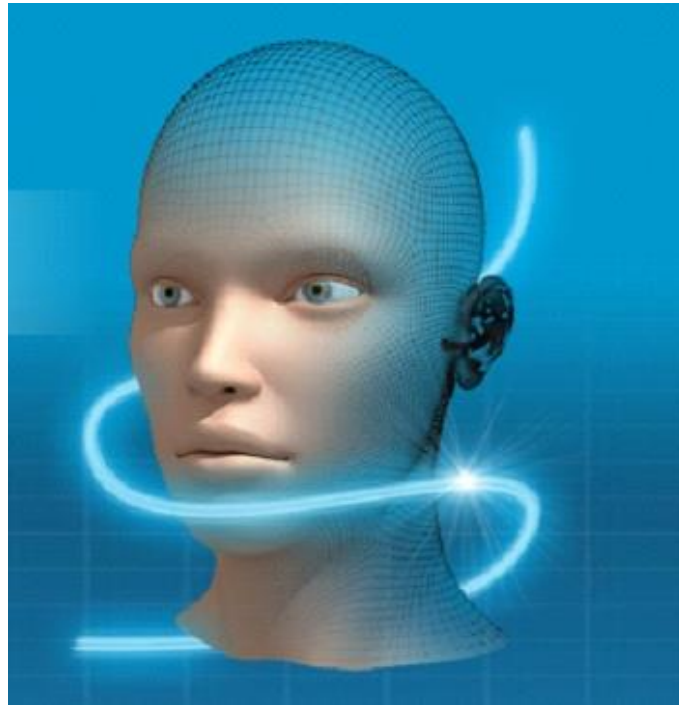


Bibliografía



- Negoita, C.V.: EXPERT SYSTEMS AND FUZZY SYSTEMS , Benjamin and Cummings, 1985.
- Parsaye, K. , Chignell, M.H.: EXPERT SYSTEMS FOR EXPERTS, John Wiley & Sons, New York, 1988.
- Rolston, D. W.: PRINCIPIOS DE INTELIGENCIA ARTIFICIAL Y SISTEMAS EXPERTOS , McGraw-Hill Latinoamericana S.A., Bogotá, 1990
- Singley, M.K., Anderson, J.R.: THE TRANSFER OF COGNITIVE SKILLS, Harvard University Press, Cambridge, 1989.
- Stylianou, A.C., Madley, G.R., Smith, R.D. : SELECTION CRITERIA FOR EXPERT SYSTEM SHELLS: A SOCIO-TECHNICAL FRAMEWORK, Communications of the ACM, vol. 35, No. 10, oct. 1992, pp. 30 - 48.
- Subrahmanyam, P. A. : *The Software Engineering of Expert Systems: Is PROLOG adequate ?* , en IEEE TRANSACTIONS ON SOFTWARE ENGINEERING , vol. SE-11, No. 11, pp. 1391-1400 , nov. 1985 .
- Walters, J.R., Nielsen, N.R.: CRAFTING KNOWLEDGE-BASED SYSTEMS, John Wiley & Sons, New York, 1988.

- Badiru, A.B.: EXPERT SYSTEMS APPLICATIONS IN ENGINEERING AND MANUFACTURING, Prentice-Hall, Englewood Cliffs, 1992.
- Bahill, A. T. : VERIFYING AND VALIDATING PERSONAL COMPUTER-BASED EXPERT SYSTEMS , Prentice-Hall, Englewood Cliffs, 1991.
- Benchimol, G., Levine, P., Pomerol, J.C.: LOS SISTEMAS EXPERTOS EN LA EMPRESA, Macrobit Editores, S.A. de C.V., México D.F. , 1990
- Boose, J.H. : EXPERTISE TRANSFER FOR EXPERT SYSTEM DESIGN , Elsevier Publishing Company , New York, 1986
- Cuena, J.: LOGICA INFORMATICA , Alianza Editorial, Madrid, 1985
- Diaper, D. (editor) : KNOWLEDGE ELICITATION: PRINCIPLES, TECHNIQUES AND APPLICATIONS, John Wiley & Sons, New York, 1989.
- Dussauchoy, A.: SISTEMAS EXPERTOS, METODOS Y HERRAMIENTAS , Paraninfo, 1988.
- Fikes, R. , Kehler, T. : THE ROLE OF FRAME-BASED REPRESENTATION IN REASONING , Communications of the ACM , vol. 28 , No. 9 , sept. 1985 , pp. 904 - 920 .
- Gallaher , J. P . : KNOWLEDGE SYSTEMS FOR BUSINESS , INTEGRATING EXPERT SYSTEMS AND M.I.S. , Prentice-Hall, Englewood Cliffs, 1988.
- Hart, A.: KNOWLEDGE ACQUISITION FOR EXPERT SYSTEMS, second edition, 196 pp., McGraw-Hill, New York, 1992.



Muchas gracias !!!

ANEXO

- Estructuras gramaticales

Encadenamiento hacia adelante - Algoritmo básico

procedimiento infiere (base de reglas, memoria de trabajo) ;

Inicializa (base de reglas, memoria de trabajo, out reglas activas);

do

conjunto conflictivo $\leftarrow$ Selecciona(memoria de trabajo, reglas activas);

if conjunto conflictivo $\neq$ then

regla $\leftarrow$ Resuelve Conflictos (conjunto conflictivo);

Dispara (regla, inout reglas activas, inout memoria de trabajo);

ActualizaEstadoCndsRegras (memoria de trabajo, inout reglas activas)

endif

until reglas activas = $\emptyset$

end

Inicialización de estructuras auxiliares

Memoria de trabajo con todos los hechos; estado de reglas y condiciones reflejan hechos en la memoria de trabajo

procedimiento **inicializa** (base de reglas, memoria de trabajo, inout reglas activas);

```
memoria de trabajo ← Recopila hechos;  
reglas activas ← base de reglas;  
for cada regla en reglas activas do  
    estado(regla) ← Activa;  
    for cond en antecedente(regla) do estado(cond) ← Libre end  
end  
ActualizaEstadoCndsRegras(memoria de trabajo, inout reglas activas);  
end
```

Función selección de reglas

```
Función selecciona (memoria de trabajo, reglas activas)  
conjunto conflictivo ←  
for cada regla en reglas activas do  
  if estado(regla) = Activa, Seleccionada then  
    conjunto conflictivo ← conjunto conflictivo  
    { regla } endif  
end devuelve(conjunto conflictivo)
```

```
Procedimiento dispara (regla, inout reglas activas)
  reglas activas ← reglas activas {regla}
  estado(regla) ← Disparada;
  atrib/val ← AtributoValor(consecuente(regla);
  memoria de trabajo ← memoria de trabajo
    {atrib/val}
end
```

procedimiento **ActualizaEstadoCndsReglas**(memoria de trabajo, inout
reglas activas)

for cada regla en reglas activas do

for cada cond en antecedente(regla) do

estado(cond) ← EvaluaCond(cond, memoria
de trabajo);

end

If evalua(antecedente(regla)) = F then

estado(regla) ← Inactiva;

reglas activas ← reglas activas {regla} endif

if evalua(antecedente(regla)) = T then

estado(regla) ← Activa, Seleccionada endif

end

Particularidades del Algoritmo visto

- Durante el procesamiento de reglas no se pregunta nada al usuario
- Se aplica una única regla del conjunto conflictivo
- El disparo de una regla altera la memoria de trabajo, entonces, el conjunto conflictivo debe ser reconsiderado en
 - cada ciclo
- El proceso se repite hasta que no existan mas reglas aplicables (se pueden definir otros criterios de terminación)
- Control del proceso de inferencia: resolución de conflictos

Estrategias de Resolución de Conflictos:

- No sensibles al contexto
 - Utilizamos un orden dado en la Base de Conocimientos para determinar una regla.
 - Utilizamos los valores de prioridades previamente establecidos.
- Sensibles al contexto
 - Se da preferencia a las reglas que involucren datos obtenidos mas recientemente.
 - Damos preferencia a las reglas mas específicas (con mas condiciones).

Se asocia una prioridad a cada regla;
Se da preferencia a las reglas con
mayor **prioridad**



Simple de programar, pero dificulta el
proceso de ingeniería del conocimiento

Resolución por Especificidad

- Se da preferencia a las reglas que mejor pormenorizan su acción, en este caso las que poseen mayor cantidad de condiciones que deban cumplirse en el antecedente.

Principales ventajas de esta estrategia:

- Comportamiento mas próximo al de un humano.
- Facilita la extensión de la base de reglas

Resolución por Actualidad

- Se da preferencia a las reglas que para ser aplicadas,
- recurren a los hechos comprobados mas recientemente
- A cada hecho se asocia una marca temporal que
- indica el “tempo” en que fue comprobado
- Ventaja: se sigue una línea única de razonamiento
- Formato para los hechos (atributos de valor único):
- Pueden aparecer diversos pares A-V en la memoria de trabajo (representando, por ejemplo, la evolución temporal del valor de una variable), pero cada triplo T-A-V es único.

Ordenamiento de las reglas del conjunto conflictivo

Conflicto:

- Se asocia a cada regla una secuencia formada por las marcas temporales de los hechos correspondientes a sus condiciones.
- Se ordenan las marcas en cada secuencia por valores decrecientes.
- Se normaliza el largo de las secuencias, aumentando con las marcas “0” que fuesen necesarias.

Características del algoritmo presentado:

- Durante el procesamiento de reglas no se pregunta nada al usuario
- Esto obliga a una recolección previa exhaustiva de datos
- Sería bueno admitir la posibilidad de solicitar al usuario datos adicionales
- Estos datos permitirían encontrar reglas adicionales para
- mejorar la calidad de las conclusiones

NUEVO ALGORITMO:

- No asume la disponibilidad inicial de todos los datos: los valores van siendo solicitados a medida de las necesidades.

Procedimiento **infiere**(base de reglas, memoria de trabajo)

Inicializa(base de reglas, memoria de trabajo, inout reglas
activas);

do

Conjunto conflictivo $\leftarrow$ Selecciona(memoria de trabajo,
reglas activas);

if conjunto conflictivo $\neq$ then

 regla $\leftarrow$ ResuelveConflictos(conjunto conflictivo);

 Dispara(regla, inout reglas activas, inout memoria de
trabajo)

Else

```
atrib ← ElijeAtributo(reglas activas); input(atrib, val);  
memoria de trabajo ← memoria de trabajo  
{atrib/val}
```

endif

ActualizaEstadoCndsReglas (memoria de trabajo, inout reglas
activas)

until reglas activas

end

Inicialización de estructuras auxiliares:

memoria de trabajo con hechos adquiridos de los usuarios (basta 1!)

Estado de reglas y condiciones reflejan hechos en la memoria de trabajo

procedimiento (base de reglas, memoria de trabajo, inout reglas activas)

memoria de trabajo $\leftarrow$ ElijeHechosIniciales



pendiente:
regla activa con
por lo menos
una condición y
otra libre

```
reglas_activas ← base de reglas;  
for cada regla en reglas activas do  
    estado(regla) ← Activa; pendiente (regla) ← F;  
    for cada cond en antecedente (regla) do  
        estado (cond) ← Libre end
```



end

```
    ActualizaEstadoCndsReglas(memoria de trabajo, inout reglas  
    activas)
```

end

procedimiento **ActualizaEstadoCndsReglas** (memoria de trabajo,
inout reglas activas)

for cada regla en reglas activas do

for cada cond en antecedente(regla) do

estado(cond) $\leftarrow$ EvaluaCond(cond, memoria de trabajo);

if estado(cond) = T then pendiente(regla) $\leftarrow$ T endif

end;

if evalua(antecedente(regla)) = F then

```
estado(regla) ← Inactiva; reglas activas ←  
reglas activas {regla};  
pendiente(regla) ← F endif;  
if evalua(antecedente(regla)) = T then  
    estado(regla) ← Activa, Seleccionada;  
    pendiente(regla) ← F endif  
end
```

Elección de Atributos

Función **Elegir Atributo** (reglas activas)

```
while
    regla ← toma una(reglas activas)
until pendiente(regla) = T OR no hay mas reglas;
if pendiente(regla) = F then regla ← toma una(reglas activas)
endif;
while
    cond ← toma un(antecedente(regla))
until estado(cond) = Libre; atrib ← atributo(cond);
end devuelve (atrib)
```